

Lựa chọn loại token (Bước 1 của quy trình)

Tài liệu này bổ sung bước còn thiếu so với quy trình chuẩn: xác định bài toán có cần token hay không, và nếu có thì chọn loại nào — trước khi viết contract.

1. Vấn đề: hợp đồng thuê hiện đang không được đại diện bởi bất kỳ token nào

Ở phiên bản đầu tiên, `RentalManager.sol` lưu toàn bộ dữ liệu thuê (property, tenant, deposit...) trực tiếp trong một `mapping` nội bộ, không có "vật đại diện quyền sở hữu" nào cho việc "đang thuê nhà A". Điều này **lệch với quy trình chuẩn** ("Cách lập trình một ứng dụng blockchain dựa trên token"), vốn yêu cầu: mọi ứng dụng dựa trên token cần xác định rõ **thứ gì được token hoá** trước khi viết contract.

2. So sánh các tiêu chuẩn token cho bài toán thuê nhà

2.1. ERC-20

Dùng cho tài sản **thay thế lẫn nhau** (1 đơn vị = 1 đơn vị khác, không phân biệt).

Không phù hợp vì:

- Một lượt thuê nhà A của người X **không thể thay thế** cho lượt thuê nhà B của người Y — mỗi hợp đồng có chủ nhà, tiền thuê, tiền cọc, thời điểm khác nhau.
- Không có cách biểu diễn "hợp đồng thuê cụ thể nào" bằng số dư token đồng nhất.

→ **Không chọn ERC-20.**

2.2. ERC-721

Dùng cho tài sản **duy nhất**, mỗi token có `tokenId` và dữ liệu riêng.

Phù hợp vì:

- Mỗi lượt thuê (một `Property` đã được một `tenant` cụ thể thuê) là **duy nhất** — giống hệt logic "mỗi chứng chỉ là một tài liệu duy nhất" trong ví dụ minh họa.
- Có thể tra cứu "ai đang giữ token nào" bằng `ownerOf(tokenId)` — chính là câu trả lời cho "ai đang thuê tài sản này".
- Không cần phát hành hàng loạt cùng lúc.

→ **Chọn ERC-721.**

2.3. ERC-1155

Dùng khi cần nhiều loại tài sản (thay thế được + không thay thế được) trong cùng một contract, hoặc phát hành số lượng lớn theo lô.

Không cần thiết ở đây vì hệ thống chỉ có một loại "tài sản" duy nhất (hợp đồng thuê), không có nhu cầu phát hành theo lô hay trộn nhiều loại token. Dùng ERC-1155 sẽ phức tạp hơn mà không mang lại lợi ích tương xứng.

→ Không chọn ERC-1155.

3. Mô hình ERC-721 không chuyển nhượng

Giống với "chứng chỉ", **hợp đồng thuê không nên chuyển nhượng được**: nếu cho phép `transferFrom`, người thuê có thể "bán" quyền thuê (và quyền đòi lại tiền cọc) cho người khác ngoài ý muốn của chủ nhà — vi phạm bản chất quan hệ thuê nhà (chủ nhà chỉ đồng ý cho một người cụ thể thuê, không phải bất kỳ ai cầm được token).

Quy tắc áp dụng trong `RentalAgreementToken.sol`:

- Chỉ được mint khi `RentalManager.rentProperty()` chạy thành công (địa chỉ được cấp `MINTER_ROLE`).
- Không cho `transferFrom` / `safeTransferFrom` / `approve` / `setApprovalForAll`.
- Không có hàm `burn` — token **giữ lại vĩnh viễn** kể cả sau khi `endLease`, làm bằng chứng lịch sử đã từng thuê (giống nguyên tắc "không xoá lịch sử" của chứng chỉ).

4. Chọn `tokenId == propertyId`

Thay vì dùng bộ đếm `tokenId` tăng dần độc lập (như ví dụ chứng chỉ dùng `_nextTokenId` riêng), hệ thống thuê nhà dùng **cùng giá trị `id`** mà `RentalManager` đã dùng cho `Property` làm `tokenId` bên `RentalAgreementToken`.

Lý do dùng được cách này mà không xung đột: trong vòng đời hiện tại, **mỗi property chỉ được thuê đúng một lần** (`Listed` → `Active` → `HandedOver` → `Ended`, không có bước "thuê lại"), nên mỗi `propertyId` chỉ được mint làm token đúng một lần duy nhất — không gian `id` không bao giờ trùng nhau.

Lợi ích: **frontend hoàn toàn không cần biết đến khái niệm "tokenId" riêng** — mọi nơi trong `ChainRentalService`, `MockRentalService`, `RentalContext`, các component đều chỉ dùng `id` như trước khi tách contract, không phải sửa gì cả.

5. `RentalManager` dùng `AccessControl` ở đúng 1 chỗ — không phải cho toàn bộ nghiệp vụ

Ví dụ minh họa (chứng chỉ) dùng `ISSUER_ROLE` / `REVOKER_ROLE` cho **toàn bộ** thao tác cấp/thu hồi, vì đó là mô hình **có cấp phép** hoàn toàn. Thuê nhà thì khác: nghiệp vụ chính (`listProperty`, `rentProperty`, `payRent`, `confirmHandover`, đề xuất/đồng ý/khiếu nại tất toán) vẫn **hoàn toàn không cấp phép** (permissionless) — bất kỳ ai cũng đăng tin/thuê được, quyền hạn xác định **bằng dữ liệu** (`msg.sender == p.landlord/tenant`), không cần admin duyệt trước.

(Cập nhật khi thêm chức năng nâng cao): sau khi bổ sung cơ chế trọng tài (multisig cho tranh chấp), `RentalManager` **giờ có dùng `AccessControl`** — nhưng chỉ cho đúng 1 vai trò `ARBITER_ROLE`, giới hạn ở đúng 1 hàm (`voteOnDispute`). Đây là ngoại lệ có chủ đích: bỏ phiếu xử lý tranh chấp buộc phải có bên được tin cậy trước (không thể để "ai cũng bỏ phiếu được"), khác hẳn với việc đăng tin/thuê nhà — vẫn giữ đúng triết lý "chỉ cấp phép ở chỗ thực sự cần một bên được tin cậy trước", không cấp phép tràn lan cho cả hệ thống.

`AccessControl` còn được dùng ở `RentalAgreementToken` — để giới hạn quyền `mintAgreement()` chỉ cho `RentalManager` gọi được (`MINTER_ROLE`), tương tự cách ví dụ chứng chỉ giới hạn quyền mint chỉ cho `CertificateManager`.

Vậy tổng cộng có **2 chỗ** dùng `AccessControl` trong toàn hệ thống: `MINTER_ROLE` (token) và `ARBITER_ROLE` (nghiep vụ trọng tài) — không phải 1 như thiết kế ban đầu, vì chức năng nâng cao trọng tài được thêm sau. Chi tiết luồng trọng tài: [gioi-han-va-rui-ro.md § 3.1](#).

6. Kết luận

Tiêu chí	Lựa chọn
Loại token	ERC-721
Chuyển nhượng	Không (khóa qua <code>_update</code> , không có <code>approve</code> /burn)
Cách sinh <code>tokenId</code>	Trùng với <code>propertyId</code> (mỗi property chỉ thuê một lần)
Ai được mint	Chỉ <code>RentalManager</code> (<code>MINTER_ROLE</code>)
<code>RentalManager</code> có <code>AccessControl</code> không	Có, nhưng chỉ 1 vai trò (<code>ARBITER_ROLE</code> , giới hạn ở <code>voteOnDispute</code>) — nghiệp vụ chính (đăng tin/thuê/trả tiền) vẫn permissionless, quyền hạn xác định bằng dữ liệu
Thư viện	OpenZeppelin Contracts 5.x (<code>ERC721</code> , <code>AccessControl</code>)